Monotonic Scheduling:

```c
#include <stdio.h>

struct task {
    int id;
    int cpu;
    int period;
};

void monotonic_scheduling(struct task t[], int n, int max_time) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (t[j].period > t[j + 1].period) {
                struct task temp = t[j];
                t[j] = t[j + 1];
                t[j + 1] = temp;
            }
        }
    }

    int remaining_time[n], next_arrival[n], misses[max_time];
    for (int i = 0; i < n; i++) {
        remaining_time[i] = 0;
        next_arrival[i] = 0;
    }
    for (int i = 0; i < max_time; i++) misses[i] = -1;

    printf("\nGantt Chart Pipeline:\n|");
    for (int time = 0; time < max_time; time++) {
        for (int i = 0; i < n; i++) {
            if (time == next_arrival[i]) {
                if (remaining_time[i] > 0) {
                    misses[time] = t[i].id;
                }
                remaining_time[i] = t[i].cpu;
                next_arrival[i] += t[i].period;
            }
        }

        int current_task = -1;
        for (int i = 0; i < n; i++) {
            if (remaining_time[i] > 0) {
                current_task = i;
                break;
            }
        }

        if (current_task != -1) {
```

```c
            printf("T%d\t|", t[current_task].id);
            remaining_time[current_task]--;
        } else {
            printf("Idle\t|");
        }
    }

    printf("\n0");
    for (int time = 0; time < max_time; time++) {
        printf("\t%d", time + 1);
    }

    printf("\n\nNotifications:");
    int has_misses = 0;
    for (int i = 0; i < max_time; i++) {
        if (misses[i] != -1) {
            printf("\n[Time %d] DEADLINE MISS: Task %d", i, misses[i]);
            has_misses = 1;
        }
    }
    if (!has_misses) printf("\nNo deadlines missed.");
    printf("\n");
}

int main() {
    int n, max_time;
    printf("Enter number of tasks: ");
    scanf("%d", &n);
    printf("Enter simulation time: ");
    scanf("%d", &max_time);

    struct task t[n];
    for (int i = 0; i < n; i++) {
        printf("Task %d (ID CPU Period): ", i + 1);
        scanf("%d %d %d", &t[i].id, &t[i].cpu, &t[i].period);
    }

    monotonic_scheduling(t, n, max_time);
    return 0;
}
```

Output:

```
wgadmin@DESKTOP-P0SNFU7 MINGW64 ~/Desktop/code/os
$ ./a.exe
Enter number of tasks: 3
Enter simulation time: 20
Task 1 (ID CPU Period): 1 3 20
Task 2 (ID CPU Period): 2 2 5
Task 3 (ID CPU Period): 3 2 10

Gantt Chart Pipeline:
|T2    |T2    |T3    |T3    |T1    |T2    |T2    |T1    |T1    |Idle  |T2    |T2    |T3    |T3    |Idle  |T2    |T2    |Idle  |Idle  |Idle  |
0      1      2      3      4      5      6      7      8      9      10     11     12     13     14     15     16     17     18     19     20

Notifications:
No deadlines missed.

wgadmin@DESKTOP-P0SNFU7 MINGW64 ~/Desktop/code/os
$ ./a.exe
Enter number of tasks: 2
Enter simulation time: 10
Task 1 (ID CPU Period): 1 3 5
Task 2 (ID CPU Period): 2 4 8

Gantt Chart Pipeline:
|T1    |T1    |T1    |T2    |T2    |T1    |T1    |T1    |T2    |T2    |
0      1      2      3      4      5      6      7      8      9      10

Notifications:
[Time 8] DEADLINE MISS: Task 2
```